PHASE

10

User Mode and System Calls

Run programs in ring 3, each in its own address space, and let them ask the kernel for services through a single, carefully guarded door

Overall progress toward Turk-OS 1.0 (this phase in red) 62 % → 70 %

Duration	3 weeks
Difficulty	★★★★★ (5/5)
You need	Phase 9: locks and wait queues; Phase 6: VMM that works on any page directory; Phase 3: GDT with ring 3 segments
You will have	A TSS, per-process page directories that share the kernel half, a first jump into ring 3, a system call gate on `int 0x80` with a dispatch table, an ELF loader for programs passed as GRUB modules, a tiny user runtime (`crt0`, syscall wrappers), pointer validation for every user buffer, and faulty programs that get killed while the kernel survives.

1 Why this phase

Everything so far has run in ring 0, where any bug can corrupt anything. An operating system's core promise is **protection**: a buggy or malicious program can crash itself, but not the kernel and not other programs. That promise rests on three pieces of hardware you have already met: privilege rings (Phase 3), paging with the U/S bit (Phase 6), and the interrupt mechanism as the only way back into ring 0 (Phase 3). This phase connects them.

You will also design the **system call interface**, the boundary between user programs and the kernel. Every file operation, every process operation and every byte of output from now on crosses it. Two habits established here protect you for the rest of the project: every argument from user space is treated as hostile, and every user pointer is checked before the kernel touches it.

2 Concepts you need

2.1 Limited direct execution

OSTEP chapter 6 frames the problem exactly. For speed, user programs should run directly on the CPU. For safety, they must not be able to do everything. The answer is to run them in **user mode**, where privileged instructions (`cli`, `hlt`, `lgdt`, `out`, loading CR3) raise #GP and pages without the U/S bit are unreachable. When a program needs something only the kernel can do, it executes a **trap** instruction that enters the kernel at an address the kernel chose at boot. On x86 that is `int 0x80` through an IDT gate whose DPL is 3, the only gate user code is allowed to invoke. MOS §1.6 walks through the steps of a `read` call, from the library wrapper down to the kernel and back.

2.2 What the CPU does on a ring change

When an interrupt or trap arrives while the CPU is in ring 3, it can't keep using the user's stack: the user controls that memory, and ESP might point anywhere. So the CPU switches to a kernel stack first, and it finds that stack in the **task state segment** (TSS): fields `ss0` and `esp0`. Then it pushes the user's SS and ESP followed by the usual EFLAGS, CS and EIP (Figure 1); these are the `useresp` and `ss` fields you left at the top of `struct regs` in Phase 3. `iret` reverses the whole thing. Turk-OS uses

the TSS only for this; x86 hardware task switching is ignored. Because each task has its own kernel stack, the scheduler must update `esp0` on every context switch.

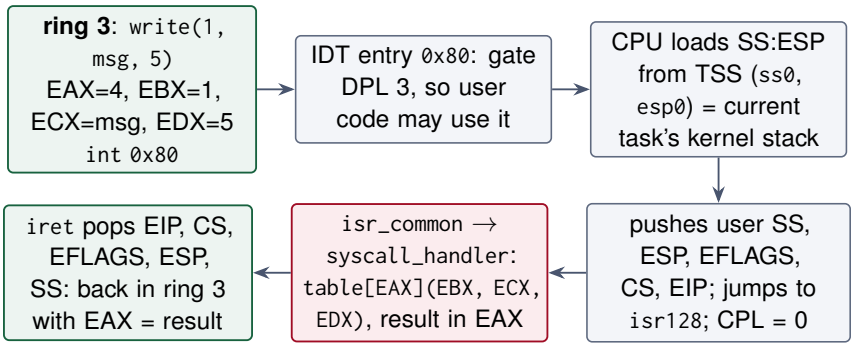


Figure 1: The round trip of a system call.

2.3 An address space per process

Each process gets its own page directory. Entries 0–767 (the user half) are private; entries 768–1023 are copied from the kernel's directory, and because you pre-allocated every kernel page table in Phase 6, those copies stay correct forever. Switching processes means loading the new directory into CR3. Figure 2 shows the layout of one process. User pages carry the U/S bit in both their PDE and PTE; kernel pages never do, so even though the kernel is mapped in every address space, user code can't touch it.

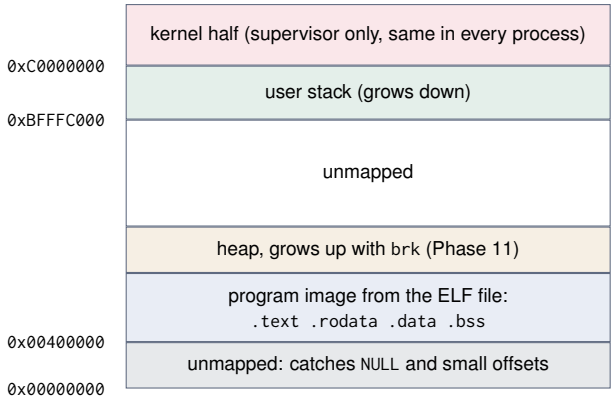


Figure 2: The address space of a Turk-OS user process.

2.4 Never trust a user pointer

When a program calls `write(1, buf, len)`, `buf` is just a number the program chose. It could point into the kernel (to trick the kernel into printing a secret), be unmapped (to crash the kernel), or wrap around the end of the address space. Every user pointer must be checked before use: the whole range lies below `0xC0000000`, doesn't overflow, and every page in it is present and user-accessible. `copy_from_user` and `copy_to_user` do those checks and return `-EFAULT` instead of faulting. MOS §9.7 shows what happens to kernels that skip this.

2.5 Loading a program

A user program is an **ELF** executable. Its header gives the entry point and a table of **program headers**, and each `PT_LOAD` header says: take `p_filesz` bytes at offset `p_offset` in the file, place them at virtual address `p_vaddr`, and zero the rest up to `p_memsz` (that tail is `.bss`). The loader allocates and maps pages, copies, zeroes, sets up a user stack, and jumps. OSC §2.5 covers linkers and loaders. Until Turk-OS has a file system (Phase 13), programs arrive as **Multiboot modules**: GRUB

(or QEMU's `-initrd`) loads extra files into memory and lists them in the Multiboot information, just as LOB chapter 7 does.

3 Reading plan

Priority	Book	What to read	Pages	Why now
REQUIRED	LOB	Ch. 7 *The Road to User Mode* (GRUB modules, executing a program); Ch. 11 *User Mode*; Ch. 13 *System Calls*	47–49, 65–68, 71–72	The x86 steps of Tasks 10.1–10.6.
REQUIRED	OSTEP	Ch. 6 §6.1–6.2 *Basic Technique*, *Restricted Operations* (re-read)	49–55	User mode, kernel mode, traps and the trap table.
REQUIRED	MOS	§1.6 *System Calls* (process, file and directory calls; the steps of `read`)	50–62	What a system call interface looks like, and a model to copy.
RECOMMENDED	OSC	§2.1–2.5 *Services*, *Interface*, *System Calls*, *Linkers and Loaders*	55–77	Parameter passing, API versus system call, loaders.
RECOMMENDED	OSC	§1.4 *Operating-System Operations* (dual mode, timers)	21–27	Why the timer is part of protection.
RECOMMENDED	OSDI	§1.4 *System Calls*	26–42	The POSIX calls a UNIX needs, in one tour.
RECOMMENDED	OSDI	§2.7 *The System Task in MINIX 3*	192–204	How MINIX lets user-space servers request privileged work.
OPTIONAL	MOS	§12.2 *Interface Design*	985–993	Principles for choosing your system calls.
OPTIONAL	MOS	§9.7 *Exploiting Software*	639–657	Why every user pointer must be checked.
OPTIONAL	OSC	§17.3 *Protection Rings*	669–671	Rings in general, and why most OSes use only two.
REQUIRED	LKD	Ch. 5, *System Calls*	69–84	Numbers, the handler, parameter passing and verifying user pointers.
OPTIONAL	LKP	Ch. 6, *Viewing the user and kernel stacks*	281–289	Two stacks per thread, seen on a running system.

Keep the Intel SDM Volume 3A at hand for the TSS format (the task management chapter) and for the stack switch on a privilege change (the interrupt chapter), and the ELF specification (*Tool Interface Standard, ELF 1.2*) for the header formats.

Real-system lens: how MINIX 3 does it

In MINIX, a POSIX call like `read` is not a kernel call at all. The library turns it into a message, and the kernel's only job is to deliver that message to a user-space server (the file system or the process manager) through `sys_call` in `kernel/proc.c (07400)`. When a server needs something privileged, such as copying data between address spaces or writing an I/O port, it sends a *kernel call* to the **system task**, `kernel/system.c (09700)`, described in OSDI §2.7. So MINIX has two layers of calls where Turk-OS has one: your `int 0x80` goes straight to a kernel function.

Linux, for comparison. LKD Linux on i386 enters system calls through `int 0x80` (or `sysenter`) into a table indexed by the number in `EAX`, and every user pointer goes through `copy_from_user` and `copy_to_user`, exactly as in Turk-OS.

4 Build it

Task 10.1 Install a TSS

Define the 104-byte TSS structure, put it in GDT entry 5 (selector `0x28`) with access byte `0x89` (present, ring 0, 32-bit available TSS) and byte granularity, and load it with `ltr`. Only `ss0` and `esp0` matter. Setting `iomap_base` to the structure's size means "no I/O permission bitmap", so any port access from ring 3 faults.

kernel/arch/i386/tss.c

```
struct tss {
    uint32_t prev_tss;
    uint32_t esp0;              /* kernel stack pointer loaded on entry from ring 3 */
    uint32_t ss0;               /* kernel stack segment: 0x10                       */
    uint32_t esp1, ss1, esp2, ss2;
    uint32_t cr3, eip, eflags, eax, ecx, edx, ebx, esp, ebp, esi, edi;
    uint32_t es, cs, ss, ds, fs, gs, ldt;
    uint16_t trap, iomap_base;
} __attribute__((packed));
_Static_assert(sizeof(struct tss) == 104, "TSS layout");

static struct tss tss;

void tss_init(void)
{
    tss.ss0 = 0x10;
    tss.iomap_base = sizeof(tss);
    gdt_set(5, (uint32_t)&tss, sizeof(tss) - 1, 0x89, 0x0);
    __asm__ volatile ("ltr %0" : : "r"((uint16_t)0x28));
}

void tss_set_kernel_stack(uint32_t esp0) { tss.esp0 = esp0; }
```

Verify: `info registers` in the QEMU monitor shows `TR=0028` with your TSS's base address.

Task 10.2 Give every process an address space

Add a page directory pointer to `struct task` (kernel threads use the kernel's directory). Write `vmm_create_address_space()`: take a page with `kalloc_page`, leave entries 0–767 empty, copy entries 768–1023 from the kernel directory. In `schedule()`, just before `switch_context`, load the next task's directory into CR3 if it differs from the current one, and call `tss_set_kernel_stack` with the top of the next task's kernel stack.

Verify: Two kernel threads with different directories both run; `info mem` shows the kernel mappings in each.

Task 10.3 The first jump into ring 3

Take a baby step before loading real programs. Create an address space, map one user page at `0x00400000` containing just the two bytes `EB FE` (`jmp $`, an infinite loop), map one user stack page, and enter ring 3 by building the frame `iret` expects: SS, ESP, EFLAGS (with IF set), CS, EIP. The user selectors have RPL 3: `0x1B` for code and `0x23` for data.

kernel/arch/i386/usermode.s

```
; void enter_user_mode(uint32_t entry, uint32_t user_esp)   -- never returns
global enter_user_mode
enter_user_mode:
    mov  ecx, [esp + 4]     ; user entry point
    mov  edx, [esp + 8]     ; user stack pointer
```

```
    mov  ax, 0x23           ; user data selector (GDT index 4, RPL 3)
    mov  ds, ax
    mov  es, ax
    mov  fs, ax
    mov  gs, ax
    push dword 0x23         ; SS
    push edx                ; ESP
    pushfd
    pop  eax
    or   eax, 0x200         ; IF = 1: interrupts stay on in user mode
    push eax                ; EFLAGS
    push dword 0x1B         ; CS (GDT index 3, RPL 3)
    push ecx                ; EIP
    iret                    ; drop to ring 3
```

Verify: The monitor stays responsive (the timer preempts the looping user task), and when you stop QEMU while that task runs, `info registers` shows `CS=001b` and `EIP=00400000`.

Task 10.4 The system call gate

Add stub 128 to `isr.s` and install it with `idt_set_gate(0x80, isr128, 0x08, 0xEF)`: a 32-bit *trap* gate with DPL 3. DPL 3 lets ring 3 use it; the trap type leaves interrupts enabled during system calls, so a long call can be preempted like any other kernel code (your Phase 9 locks make that safe). Register a handler that dispatches on EAX. Use the Linux i386 numbers for the calls you implement; then any Linux reference table matches yours.

kernel/syscall/syscall.c

```
enum { SYS_exit = 1, SYS_read = 3, SYS_write = 4, SYS_getpid = 20,
       SYS_sched_yield = 158, SYS_nanosleep = 162, NR_SYSCALLS = 256 };

typedef int32_t (*syscall_fn)(uint32_t a, uint32_t b, uint32_t c);

static const syscall_fn syscall_table[NR_SYSCALLS] = {
    [SYS_exit]        = sys_exit,
    [SYS_read]        = sys_read,
    [SYS_write]       = sys_write,
    [SYS_getpid]      = sys_getpid,
    [SYS_sched_yield] = sys_yield,
    [SYS_nanosleep]   = sys_sleep_ms,      /* simplified: argument is milliseconds */
};

static void syscall_handler(struct regs *r)
{
    if (r->eax >= NR_SYSCALLS || !syscall_table[r->eax]) {
        r->eax = (uint32_t)-ENOSYS;
        return;
    }
    r->eax = syscall_table[r->eax](r->ebx, r->ecx, r->edx);   /* return value to user */
}
```

Portability note

`syscall_handler` is generic code, yet it reads x86 registers by name. Dispatch through three one-line inline helpers in the x86 header instead, such as `tf_sysno(r)`, `tf_arg(r, n)` and `tf_set_ret(r, v)`: Phase 16 makes exactly these part of the interface every architecture provides. The ports keep these Linux i386 call numbers and change only the trap: RISC-V uses `ecall` with the number in `a7` and the arguments in `a0`–`a5`, AArch64 uses `svc #0` with the number in `x8` and the arguments in `x0`–`x5`, and the result comes back in `a0` or `x0`.

Task 10.5 A user runtime

Create `user/` with its own linker script (programs start at `0x00400000`), a start file that calls `main` and passes its return value to `exit`, and syscall wrappers in inline assembly. Build each program as a separate ELF with the same cross compiler, linked with `-nostdlib` and `-T user/user.ld`.

user/crt0.s

```
global _start
extern main
extern exit
section .text
_start:
    call main               ; argc and argv arrive in Phase 11
    push eax
    call exit               ; exit(main's return value), never returns
```

user/lib/syscall.c

```
static inline int32_t syscall3(uint32_t n, uint32_t a, uint32_t b, uint32_t c)
{
    int32_t ret;
    __asm__ volatile ("int $0x80" : "=a"(ret) : "a"(n), "b"(a), "c"(b), "d"(c) : "memory");
    return ret;
}

int  write(int fd, const void *buf, unsigned len) { return syscall3(4, fd, (uint32_t)buf, len); }
int  getpid(void)                                 { return syscall3(20, 0, 0, 0); }
void exit(int code)                               { syscall3(1, code, 0, 0); for (;;) { } }
```

Task 10.6 Load programs as boot modules

Add `module /boot/hello.elf` lines to `grub.cfg`. For quick runs, QEMU's Multiboot loader takes the modules as a comma-separated `-initrd` list:

terminal

```
qemu-system-i386 -kernel build/kernel.elf -serial stdio \
    -initrd "build/user/hello.elf,build/user/evil.elf"
```

In the kernel, walk `mods_count` and `mods_addr` (through `P2V`), reserve the modules' frames in the PMM (you did that in Phase 5), and keep a table of name, start and end for the loader.

Task 10.7 The ELF loader

Write `elf_load(pgdir, image, size, &entry)`. Validate before trusting anything: the magic bytes `7F 45 4C 46`, 32-bit class, little-endian, type `ET_EXEC` (2), machine `EM_386` (3), and program headers that lie inside the file. For each `PT_LOAD`, check that the segment lies entirely below `0xC0000000` without overflow, allocate and map user pages (writable only if the segment's flags say so), copy `p_filesz` bytes through the direct map, and zero the rest up to `p_memsz`. Then map a user stack below `0xC0000000`, create a task that runs `enter_user_mode(entry, stack_top)`, and make it ready.

include/elf.h

```
struct elf32_ehdr {
    uint8_t  e_ident[16];          /* 0x7F 'E' 'L' 'F', class, data, version ... */
    uint16_t e_type, e_machine;
    uint32_t e_version, e_entry, e_phoff, e_shoff, e_flags;
    uint16_t e_ehsize, e_phentsize, e_phnum, e_shentsize, e_shnum, e_shstrndx;
} __attribute__((packed));

struct elf32_phdr {
```

```
    uint32_t p_type;              /* 1 = PT_LOAD                  */
    uint32_t p_offset, p_vaddr, p_paddr;
    uint32_t p_filesz, p_memsz;     /* memsz > filesz: zero the tail */
    uint32_t p_flags, p_align;      /* flags: 1 = X, 2 = W, 4 = R    */
} __attribute__((packed));
```

Verify: A `hello` program prints "Merhaba from ring 3, pid 3" through `write`, then exits, and its pages go back to the PMM (free count restored).

Task 10.8 Guard the door

Write `copy_from_user`, `copy_to_user` and `strncpy_from_user`, and route every system call argument that is a pointer through them. `sys_write` copies the user buffer in chunks into a kernel buffer before printing. Then make exceptions depend on who caused them: if `(r->cs & 3) == 3`, the fault came from user mode, so print something like "hello (pid 3): segmentation fault at 0x00000000" and kill that task; only faults in ring 0 still panic.

kernel/syscall/uaccess.c

```c
int copy_from_user(void *dst, const void *usrc, size_t n)
{
    uintptr_t start = (uintptr_t)usrc, end = start + n;
    if (end < start || end > KERNEL_VMA)              /* wraps around, or reaches the kernel */
        return -EFAULT;
    for (uintptr_t pg = start & ~0xFFFu; pg < end; pg += PAGE_SIZE)
        if (!vmm_user_page_ok(current->pgdir, pg))    /* present and U/S set in PDE and PTE */
            return -EFAULT;
    memcpy(dst, usrc, n);                             /* current address space is active */
    return 0;
}
```

Task 10.9 Write evil programs

A protection mechanism is only as good as its tests. Write user programs that each try one thing they must not do, load them all at once, and check that each is killed or gets an error while `hello` and the kernel carry on.

Program	What it tries	Expected result
`divzero`	divide by zero	killed (#DE from ring 3)
`nullptr`	read address 0	killed (page fault, user mode)
`kpeek`	read `*(int *)0xC0100000`	killed (protection violation)
`cli`	execute `cli` or `hlt`	killed (#GP)
`badptr`	`write(1, (void *)0xC0000000, 16)`	`write` returns `-EFAULT`
`hugelen`	`write(1, buf, 0xFFFFFFFF)`	`-EFAULT` (range wraps around)
`badsys`	`int 0x80` with EAX = 9999	returns `-ENOSYS`

Verify: All seven behave as listed in a single boot, and `ps` afterwards shows only the kernel's own tasks.

5 Testing and debugging

Symptom	Likely cause	What to do
#GP on the `iret` into ring 3	Selectors without RPL 3, or GDT entries 3–4 with the wrong DPL	Use `0x1B` and `0x23`; check access bytes `0xFA` and `0xF2`.

Symptom	Likely cause	What to do
Page fault at the user entry point with error code 5 or 7 (user mode, protection violation)	The PDE or the PTE lacks the U/S bit	Both levels need it; print both entries for the faulting address.
Triple fault on the first interrupt or syscall from ring 3	TSS not loaded, `esp0` zero, or the TSS descriptor is wrong	Check `TR` in `info registers`; set `esp0` before entering ring 3.
User program never gets preempted	EFLAGS pushed for `iret` without IF	OR `0x200` into EFLAGS in `enter_user_mode`.
Second process crashes after a context switch	CR3 switched, but `esp0` still points at the first task's stack	Update `esp0` and CR3 together, on every switch.
Kernel crashes inside a system call on a bad pointer	A pointer was used directly instead of through `copy_from_user`	Search for every user pointer dereference; there should be none outside `uaccess.c`.

6 Milestone

Milestone: Protection works

- ☐ The TSS is loaded, and `esp0` and CR3 are updated on every context switch.
- ☐ Each process has its own page directory sharing the kernel half.
- ☐ ELF programs from boot modules run in ring 3 and use `write`, `getpid`, `sleep` and `exit`.
- ☐ All user pointers pass through `copy_from_user/copy_to_user`.
- ☐ All seven evil programs are contained; the kernel never panics because of user code.
- ☐ `docs/syscalls.md` lists each call: number, arguments, return value, errors.

Self-check questions

1. List, in order, everything the CPU does when a ring-3 program executes `int 0x80`.
2. Why does the TSS need `esp0`, and why must it change on every context switch?
3. Why is the system call gate a trap gate with DPL 3, while exception gates are interrupt gates with DPL 0?
4. How does a system call's return value get back to the user program?
5. Why must both the PDE and the PTE have the U/S bit set? What happens if only the PTE has it?
6. What does `copy_from_user` check, and what could a program do if it didn't?
7. The kernel is mapped in every address space. Why can't user code read it?

7 Going further

Replace `int 0x80` with the faster `sysenter/sysexit` pair and measure the difference with `rdtsc`. Put `argc`, `argv` and the environment on the initial user stack in the System V i386 layout, so `main(int argc, char **argv)` works (Phase 11 needs this for `exec` anyway). Add a read-only page shared by all processes that holds the current tick count, so programs can read the time without a system call; Linux's vDSO grew from the same idea.

Next: **Phase 11 Process API: fork, exec, wait and exit** →

Books used in this plan

LOB Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at https://littleosbook.github.io.

OSTEP Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at https://pages.cs.wisc.edu/~remzi/OSTEP/.

OSC A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.

The books do not come with this plan. **LOB** and **OSTEP** are free online; buy or borrow the other five (the roadmap lists their ISBNs). Where books disagree, the plan follows the newer one when its method is also better for Turk-OS. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free **OSTEP** chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in **OSDI** Appendix B.